

Aperture — Database Design (Part 2)

Version: v1.0

Document Type: Database Design

20. Identity Domain

Purpose

The Identity domain stores authentication-related information.

This domain is intentionally separated from the Users domain.

Authentication proves who someone is.

Users defines who they are within Aperture.

Entity: Identity

Description

Represents a login identity.

Supports:

- Email/password
- Google OAuth

Future:

- Apple OAuth
 - GitHub OAuth
 - Passkeys
-

Key Attributes

- UUID
- Email
- Password Hash

- Authentication Provider
 - Provider Identifier
 - Account Status
 - Email Verification Status
 - Created At
 - Updated At
-

Relationships

Identity



One User

Design Rationale

Separating Identity from User allows future authentication changes without affecting the rest of the platform.

Entity: Refresh Session

Represents an authenticated device/session.

Stores:

- Session Identifier
- Refresh Token Identifier
- Device Metadata (future)
- Expiration
- Revocation Status

Multiple refresh sessions may exist per Identity.

21. Users Domain

Entity: User

Purpose

Represents a person within Aperture.

Key Attributes

- UUID
 - Username
 - Display Name
 - Biography
 - Avatar URL
 - Profile Visibility
 - Created At
 - Updated At
-

Relationships

User

↓

Many Reviews

↓

Many Lists

↓

Many Ratings

↓

Many Notifications

↓

Many Follow Relationships

Design Decisions

The User table intentionally excludes authentication data.

This reduces coupling between authentication and community features.

Entity: User Preferences

Purpose

Stores user-specific configuration.

Examples:

- Theme
- Language
- Notification Preferences
- Spoiler Preferences
- Preferred Region

Keeping preferences separate prevents the User table from becoming excessively wide.

Entity: Follow

Represents:

User A follows User B.

Self-follow relationships are prohibited.

Duplicate relationships are prevented through a unique constraint.

22. Metadata Domain

This is the largest domain.

Entity: Movie

Purpose

Canonical representation of a film.

Key Attributes

- UUID
- TMDb Identifier

- Title
- Original Title
- Release Date
- Runtime
- Overview
- Status
- Poster
- Backdrop

Relationships

Movie



Genres



Cast



Crew



Collections



Reviews



Lists



Streaming Availability

Movies never store reviews directly.

Relationships remain normalized.

Entity: TV Show

Very similar to Movie.

Additional relationships:

↓

Seasons

↓

Episodes

Entity: Season

Represents one season of a television show.

Stores:

- Season Number
 - Episode Count
 - Poster
 - Overview
-

Entity: Episode

Stores:

- Episode Number
- Runtime
- Air Date
- Overview

Episodes inherit ownership through the parent TV show.

Entity: Person

Represents:

- Actors
- Directors
- Writers
- Producers

Stored once.

Referenced many times.

Entity: Genre

Reference data.

Examples:

- Drama
- Thriller
- Animation

Stored independently to avoid duplication.

Entity: Collection

Represents franchises.

Examples:

- Lord of the Rings
- Harry Potter
- Toy Story

Collections own ordering.

Movies reference collections.

Entity: Studio

Represents production companies.

Allows filtering by studio.

Useful for discovery.

23. Review Domain

Entity: Rating

Purpose

Numerical opinion.

Separate from Review.

Reason:

Many users rate without writing reviews.

Relationships

User

↓

Rating

↓

Movie

Entity: Review

Purpose

Long-form opinion.

Key Attributes

- Rating Reference
- Review Body

- Spoiler Flag
 - Visibility
 - Edit Timestamp
-

Relationships

Review

↓

Likes

↓

Future Comments

Entity: Review Like

Simple association table.

Tracks:

User

↓

Liked Review

Unique constraint prevents duplicate likes.

24. Lists Domain

Entity: List

Represents curated collections.

Examples:

- Watchlist
- Favorites
- Top 100 Sci-Fi

Key Attributes

- Owner
- Title
- Description
- Visibility

Relationships

List



Many List Items

Entity: List Item

Stores:

- List Reference
- Movie/TV Reference
- Position
- Date Added

Ordering belongs to the List Item rather than the List.

This simplifies reordering operations.

25. Streaming Domain

Entity: Streaming Provider

Reference table.

Examples:

- Netflix
- Prime Video
- Disney+

Stored once.

Entity: Availability

Represents

Movie X

↓

Netflix

↓

India

↓

Subscription

Availability changes frequently.

The table is optimized for updates rather than historical storage.

26. Notification Domain

Entity: Notification

Represents one user-facing notification.

Stores:

- Recipient
- Type
- Payload
- Read Status
- Created At

Payload should remain lightweight.

Entity: Notification Preference

One record per user.

Defines:

- Enabled Categories
- Delivery Channels
- Digest Preferences (future)

27. Administration Domain

Entity: Audit Log

Immutable.

Stores:

- Administrator
- Action
- Target
- Timestamp
- Result

Critical for security.

Entity: Feature Flag

Allows controlled rollout of features.

Examples:

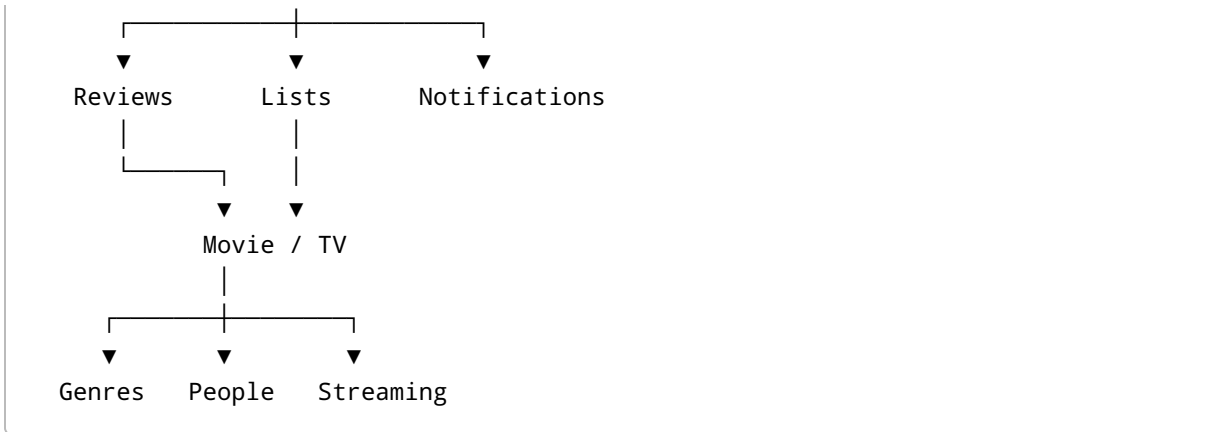
- Semantic Search
- AI Recommendations
- Movie Spaces

Configuration should be data-driven rather than code-driven.

28. Relationship Overview

The primary relationship graph is intentionally centered around three core entities.

User
|



Most other entities are either reference data or supporting entities.

29. Schema Design Principles

The schema follows several principles.

Reference data is normalized.

Examples:

Genres

Studios

Providers

High-volume content is separated.

Examples:

Reviews

Ratings

Notifications

Frequently changing data is isolated.

Examples:

Streaming Availability

Notification State

Refresh Sessions

Static metadata is centralized.

Examples:

Movies

TV Shows

People

Collections

This separation allows different parts of the system to evolve independently.

30. Summary

The Aperture schema is organized around domain ownership rather than technical convenience.

Each entity has a single owner, clear relationships, and a well-defined purpose.

This structure minimizes duplication, preserves referential integrity, and provides a stable foundation for future features such as semantic search, AI recommendations, collaborative lists, and community spaces.